

RELATÓRIO DE ELABORAÇÃO DO PROJETO

Título do Projeto: Period tracker em Python

Data: 19 de junho de 2026

Autor: Carmen Gomes

SUMÁRIO

1. Introdução
2. Descrição do Projeto
3. Levantamento de Requisitos
4. Análise de Sistemas
5. Design do Sistema
6. Implementação
7. Testes
8. Conclusão

1. INTRODUÇÃO

Contextualização

O ciclo menstrual tem um impacto enorme no nosso dia a dia, influenciando o nosso bem-estar, energia e humor. A ideia para este projeto surgiu no âmbito da unidade de Projeto de Programação do curso de Python do IEFP, onde decidi desenvolver uma versão mais simplificada de uma ideia que já tinha em mente. Em vez de criar algo com a complexidade excessiva das aplicações comerciais, o meu objetivo foi desenhar uma solução interativa e direta via terminal. Assim, o programa não serve apenas para registar dados, mas ajuda também a utilizadora a perceber, de forma simples, o que se passa com o seu corpo em cada fase do ciclo.

Objetivo do Relatório

Este documento serve para registar de forma clara todas as fases de planeamento, arquitetura, desenvolvimento técnico e validação do sistema *Period Tracker*, apresentando as escolhas de design de código e a persistência de ficheiros adotadas.

2. DESCRIÇÃO DO PROJETO

Visão Geral

A minha ideia para o projeto de programação é fazer um programa básico que ajude mulheres a entender o que se passa no seu corpo, armazenando o histórico dos seus ciclos menstruais e recomendando profissionais com base nos seus sintomas mais registados. Confesso que seria uma versão mais simplificada de apps existentes, e que para além de mostrar o porquê da provável aparição de cada sintoma, o seu “diferencial” seria recomendar diretamente especialistas que possam ajudá-las durante o ciclo.

Escopo

- **Incluso:** Configuração interativa de perfil, cálculo dinâmico da fase por matemática de datas, loop iterativo contínuo para inserção em cadeia de sintomas diários das 4 fases, alertas clínicos por cores ANSI e recomendação de especialistas cruzando sintomas e geolocalização por ficheiro externo.
- **Não Incluso:** Autenticação complexa por palavra-passe baseada em SQL nesta versão, interface gráfica web e processamento de dados médicos em tempo real fora da árvore de decisão local.

Objetivos

- Ajudar as utilizadoras a monitorizar os 28 dias do ciclo sem necessidade de fechar a aplicação repetidamente.
- Oferecer um sumário informativo no início de cada fase do ciclo menstrual.
- Emitir avisos visuais imediatos (com cores dedicadas no terminal) caso os sintomas fujam dos parâmetros saudáveis padrão.

3. LEVANTAMENTO DE REQUISITOS

Metodologia de Recolha

Os requisitos foram definidos com base em rotinas de saúde feminina, dividindo o ciclo de 28 dias nas suas 4 janelas biológicas gerais: Menstruação, Fase Folicular, Ovulação e Fase Lútea.

Requisitos Funcionais

1. **Configuração de Perfil:** No primeiro arranque, perguntar o nome, idade e a data da última menstruação (AAAA-MM-DD), gravando imediatamente num ficheiro JSON.
2. **Mensagens de Boas-vindas Dinâmicas:** Se os dados já existirem, saudar a utilizadora pelo nome; caso contrário, disparar a configuração inicial.
3. **Identificação Fluxo:** Se for o dia 1 ou o dia anterior tiver o rótulo de "Menstruação", perguntar ativamente se o período continua. Caso a resposta mude para "Não", saltar essa verificação nos dias seguintes para otimizar a experiência.
4. **Menus de Sintomas por Numeração:** Permitir a escolha do fluxo (Pesado/Médio/Ligeiro), cólicas (Pesadas/Medianas/Ligeiras/Nenhuma) e uma lista múltipla de queixas diárias via loop while.
5. **Frases Introdutórias:** Exibir um resumo sobre o que está a acontecer no organismo imediatamente antes de solicitar os sintomas do dia.
6. **Análise de Saúde:** Disponibilizar uma opção no menu para analisar os dados inseridos, contar anomalias (como sangramento por mais de 7 dias ou múltiplos dias de vômitos) e acionar alertas visuais.
7. **Filtro de Médicos com base na localização:** Perguntar a localização (Lisboa/Porto) caso haja interesse, guardá-la no perfil e puxar uma lista de ginecologistas ou clínicos gerais associados àquela zona específica.

Requisitos Não Funcionais

1. **Linguagem de execução:** O sistema tem de ser integralmente desenvolvido em Python 3.
2. **Persistência:** O armazenamento dos dados tem de ocorrer em ficheiros JSON salvos exclusivamente dentro do projeto, permitindo a retenção contínua.
3. **Interface baseada em terminal:** O texto deve estar bem formatado no terminal, recorrendo a cores ANSI para destacar as ações (Introdução de dados - Amarelo, Alertas Graves - Vermelho, Sucesso/Ok - Verde e Notas Informativas - Azul).

4. ANÁLISE DE SISTEMAS

Modelagem do Fluxo de Trabalho

O ciclo de execução respeita uma ordem sequencial controlada por um loop principal que impede a quebra do programa até indicação em contrário:

[Início do Programa] -> Carrega Ficheiro JSON

|
v
[Perfil Vazio?] -----(Sim)-----> [Configura Perfil: Nome, Idade, Data] -> Grava JSON

|
(Não) |
|<-----/v

[Menu Principal]

|---> Opção 1: Registrar Sintomas do Dia -> Verifica fase anterior -> Guarda Sintomas ->
Loop Continua

|---> Opção 2: Ver Relatório de Saúde --> Conta desvios -> Ativa Alertas ANSI -> Filtra
Médicos por Zona

|---> Opção 3: Sair da Aplicação -----> Grava ficheiros de forma segura e encerra o
processo

5. DESIGN DO SISTEMA

Arquitetura do Sistema

O projeto foi estruturado seguindo o princípio da responsabilidade única e modularidade, dividindo as funcionalidades por módulos independentes que interagem entre si, isolando a zona de desenvolvimento da zona de validação de código:

- src/ler_gravar.py: Camada de dados responsável por gerir a leitura e escrita física de ficheiros estruturados no disco.
- src/ciclo.py: Lógica de recolha de questionários, cálculo de datas e armazenamento de mensagens informativas das fases.

- `src/saude.py`: Motor de análise clínica e triagem estatística encarregue de cruzar sintomas e geolocalização.
- `src/main.py`: Orquestrador central e gestor dos menus e loops de controlo da aplicação.

Interface do Utilizador (Formatação de Terminal)

A interface foi projetada utilizando esquemas de cores específicos para garantir uma leitura rápida no terminal:

- \033[91m (Vermelho): Utilizado para alertas de segurança de saúde máxima (ex: sangramento superior a 7 dias consecutivas).
- \033[93m (Amarelo): Utilizado para sinalizar títulos de menus, inserção ativa de dados ou avisos preventivos moderados.
- \033[92m (Verde): Indica dados validados com sucesso, parâmetros normais ou destaques de localização.
- \033[94m (Azul): Cor exclusiva utilizada nas introduções científicas no início de cada recolha de sintomas para educar a utilizadora.

6. IMPLEMENTAÇÃO

Tecnologias e Ferramentas Utilizadas

- **Linguagem de programação:** Python 3
- **Bibliotecas nativas:** json para tradução de tabelas de dados, pathlib para controlo absoluto e independente de caminhos de ficheiros e datetime para cronometragem precisa.
- **Ambiente de desenvolvimento:** Visual Studio Code.
- **Controlo de versões:** Git local para registo histórico e incremental de etapas de código.

Estrutura e Organização de Pastas

Plaintext

projeto-ufcd-10790/

```

|
|— src/                                # Pasta com o código de produção
|   |— __init__.py                    # Sinalizador de pacote Python
|   |— ler_gravar.py                 # Leitor e gravador de JSON
|   |— ciclo.py                      # Questionários e lógica das fases
|   |— saude.py                      # Motor de regras, cores ANSI e contactos
|   |— main.py                      # Orquestrador do menu principal e loop contínuo
|   |— dados_ciclo.json              # Ficheiro gerado com o perfil e sintomas diários
|   |— medicos.json                  # Catálogo fixo de médicos por localização
|
|— tests/                             # Pasta isolada de desenvolvimento e testes
|   |— __init__.py
|   |— teste_ler_gravar.py           # Validador inicial da persistência de dados
|   |— teste_interativo.py          # Simulador de respostas diárias em ambiente de testes

```

7. TESTES

Plano de Testes Modulares

- **Teste Unitário de Persistência (tests/teste_ler_gravar.py):** Focado em simular o comportamento da aplicação numa primeira execução. Caso o ficheiro de dados não exista na pen drive ou diretório, valida se o sistema inicializa a estrutura base em falta de forma limpa, gravando e alterando dados logo a seguir.
- **Teste Interativo de Componentes (tests/teste_interativo.py):** Criado para isolar as perguntas interativas do ecrã principal de produção. Permite simular no terminal a recolha contínua do dia do ciclo, injetar esses sintomas no dicionário e validar se a leitura subsequente recupera as chaves corretamente.

Casos de Teste e Resultados Práticos

1. **Caso de Teste 1 — Inicialização do Perfil:** Correr a aplicação sem ficheiro JSON prévio.
 - *Resultado:* Sucesso. O sistema disparou o configurador, recolheu os dados e criou o ficheiro dados_ciclo.json com sucesso na pasta correta.
2. **Caso de Teste 2 — Avanço do Loop:** Registrar sintomas do Dia 1 e reiniciar a aplicação.
 - *Resultado:* Sucesso. O programa leu o histórico, identificou que já existia um dia preenchido e abriu automaticamente o menu numerado focado no Dia 2 do ciclo.
3. **Caso de Teste 3 — Otimização de Pergunta de Fluxo:** Responder que "Não" está menstruada no Dia 5.
 - *Resultado:* Sucesso. Ao avançar para o Dia 6, o programa espreitou o índice [-1] do histórico do JSON, verificou que a fase anterior já era sem sangramento, e saltou imediatamente para as perguntas gerais sem questionar redundâncias.
4. **Caso de Teste 4 — Triagem Clínico-Geográfica (Simulação de Anomalia):** Inserir 8 dias seguidos de fluxo pesado e sintomas de vômitos em Lisboa.
 - *Resultado:* Sucesso. Ao pedir o relatório de saúde, o sistema ativou com sucesso as cores vermelhas de alerta no terminal e imprimiu as duas listas de médicos correspondentes separadas (Ginecologia e Clínica Geral), extraídas do ficheiro local.

8. CONCLUSÃO

Resumo dos Resultados

O desenvolvimento do *Period Tracker* atingiu todos os objetivos que tinha definido para este projeto de programação. Conseguimos criar uma aplicação muito limpa e fácil de usar, com menus numéricos simples que protegem o funcionamento do nosso ficheiro JSON. O programa revelou-se super flexível, dando total liberdade à utilizadora para registar os dias reais da sua menstruação. Para além disso, mostrou-se inteligente ao cruzar os sintomas com os avisos de saúde e a recomendação de médicos por zona.

Lições Aprendidas e Reflexão

A execução prática deste projeto ensinou-me a importância de estruturar um programa de forma modular. Isolar as pastas e usar o comando `python3 -m` garantiu que o Python nunca perdesse os caminhos e as importações de ficheiros. Também enfrentei alguns desafios técnicos, como o erro ao tentar usar listas dentro dos dicionários, o que me obrigou a entender o uso de loops for para extrair os dados sem o programa crashar. No final, o projeto ficou com uma base de código sólida, organizada e pronta para futuras melhorias.

Referências:

-  Cópia de Requisitos Grelha Template.xlsx
-  Cópia de Gantt_Template.xlsx
-  1 - Como Construir um Relatório de Elaboração do Projeto.pdf
-  2 - Exemplo Relatorio de Projeto.pdf